

CeylonPick

Technical Architecture & Build Plan

Domain model · Module boundaries · Order state machine · Ledger · Dashboards · UX patterns · 8-week plan

Version v1.0 · 5 September 2026 · Status: Baseline (approved for build)

Version History

Version	Date	Author	Summary of changes
v1.0	5 September 2026	Founder (with AI-assisted drafting)	Initial baseline: context, stack, modules, domain model, order state machine, ledger design, attribution, integrations, dashboards, UX patterns, NFRs, 8-week plan, ADRs.
v1.1	—	—	(reserved) Post day-90 review: update state machine and ledger with real courier/PayHere behaviour.
v2.0	—	—	(reserved) Campaign Board, creator dashboard v2, reviews, referral codes.

How to use this document

Sections 1–4 explain **what** is being built and why the shape is deliberately small. Sections 5–9 are the engineering contract: domain model, state machine, ledger and integration rules that must not be violated even under time pressure. Sections 10–11 cover the dashboards and the user-experience patterns. Section 12 is the 8-week plan; Section 13 records the architectural decisions so they are not re-argued every week.

Companion documents: *CeylonPick Feasibility Study* and *CeylonPick Project Plan v1.0* (business scope, budget, KPIs, kill criteria).

1. Purpose, Scope and Constraints

This document defines the technical architecture for CeylonPick v1 — a curated marketplace for verified Sri Lankan-made brands with creator (affiliate) storefronts. It exists to let a single developer-founder ship a correct, money-safe system in eight weeks and then grow it for twelve months on a LKR 4,000–6,000 monthly infrastructure budget.

Hard constraints

- **One developer**, AI-assisted. Anything that needs a second on-call person is out of scope.
- **Infra budget ≤ LKR 6,000/month** in year 1.
- **Money correctness over features**. Held funds, vendor payables and creator payables must always reconcile to the bank statement.
- **COD is first-class** (~55–65% of orders). OTP confirmation and RTO handling are core paths, not edge cases.
- **Mobile-first**. 85%+ of traffic will be phones on variable bandwidth.
- **8-week timebox**. If v1 is not live by week 8, switch to WooCommerce + Dokan and ship. The business does not wait for the code.

Explicitly out of scope for v1

Native mobile apps, microservices, Redis/message brokers, search engines (Elasticsearch/Meilisearch), multi-currency, international shipping, vendor self-onboarding without manual verification, news/jobs/general content sections, clothing and electronics categories.

2. System Context

Four human actors and four external systems. Everything else is internal to the monolith.

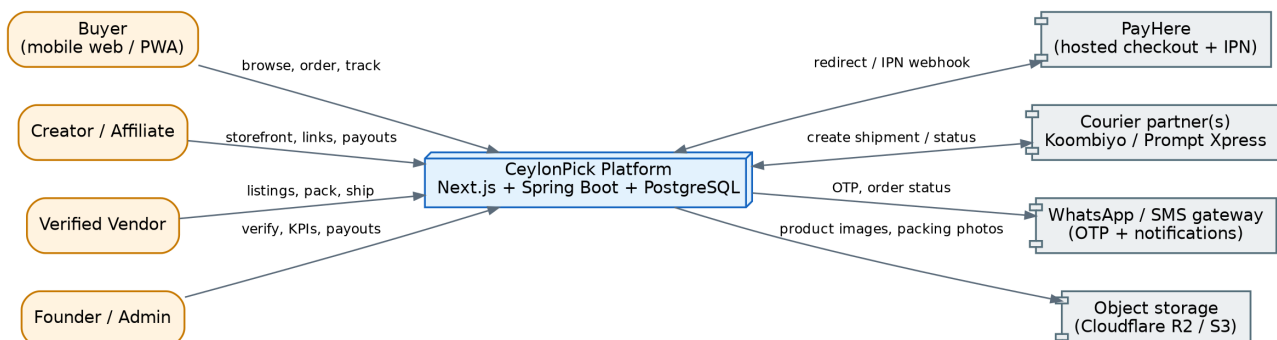


Figure 1 — System context. The platform is the single integration point for payments, courier, messaging and storage.

Actor / System	Role	Key interactions
Buyer	Purchases via mobile web/PWA, mostly via a creator link	Browse, checkout (PayHere or COD+OTP), track order, raise dispute (≤48h)
Creator	Affiliate with a storefront page and codes	Apply, get links/codes, see clicks → confirmed → delivered → payable, receive payouts
Vendor	Verified maker	Manage listings/stock, pack with photo, hand to courier, see payables and payout history
Founder / Admin	Operates the business	Verify vendors/creators, monitor KPIs and RTO, run payouts, resolve disputes
PayHere	Hosted card/wallet/bank checkout	Redirect checkout; server-to-server IPN (notify_url) with MD5 signature; settlement report
Courier	Koombiyo / Prompt Xpress / Pronto (adapter per courier)	Create shipment, receive status (webhook or polling), COD remittance report import
Messaging	WhatsApp Business API or local SMS gateway (Notify.lk / Text.lk / Dialog)	OTP, order status, payout notices

Actor / System	Role	Key interactions
Object storage	Cloudflare R2 (S3-compatible)	Product images, packing photos, nightly DB dumps

3. Architecture Overview

A **modular monolith**: one Spring Boot deployable with strict internal module boundaries, one PostgreSQL database, and a separately deployed Next.js front end. This gives microservice-style separation of concerns without distributed-system cost. Modules communicate through in-process domain events written to a transactional outbox table; no message broker is used.

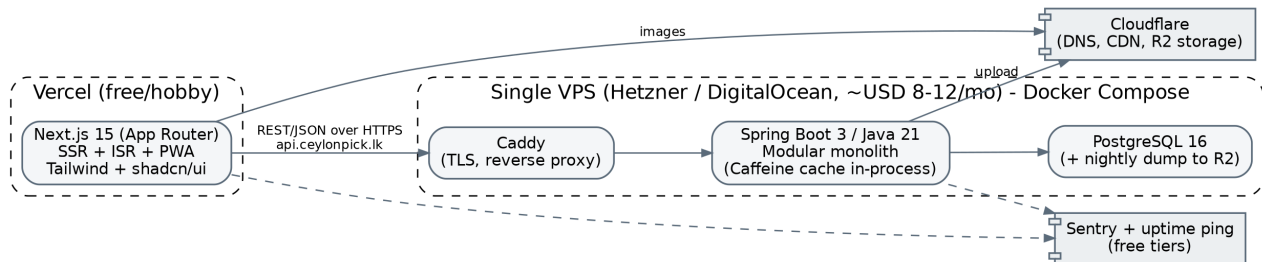


Figure 2 — Deployment view. Total hosting cost ≈ USD 10–15/month (≈ LKR 3,500–5,000).

Layer	Choice	Why (and what would change the decision)
Front end	Next.js 15 (App Router), TypeScript, Tailwind, shadcn/ui, next-pwa	SSR/ISR for SEO'd product pages; PWA installability; React ecosystem for dashboards. Change if: never — this is stable.
Back end	Spring Boot 3.x, Java 21, Spring Modulith (optional), Spring Data JPA, Flyway	Founder's strength; strong typing for money logic; Modulith enforces package boundaries. Change if: solo velocity suffers → Kotlin is a drop-in.
Database	PostgreSQL 16	Transactions, JSONB for addresses/attrs, row-level locking for ledger, LISTEN/NOTIFY not needed. Never change.
Cache	Caffeine (in-process) + Next.js ISR	Catalog reads are 95% of traffic and change rarely. Add Redis only at 2+ app instances or >1,000 orders/day.
Jobs	Spring @Scheduled + DB-backed outbox/job table (ShedLock for single-run safety)	7-day settlement, OTP expiry, IPN reconciliation, courier polling. Change if: job volume > ~10k/day → Quartz or a queue.
Auth	Spring Security, JWT in httpOnly cookie, roles BUYER/CREATOR/VENDOR/ADMIN; buyer login via phone OTP (passwordless)	Buyers should never need a password. Vendors/creators: email+password + optional OTP.
Storage	Cloudflare R2 via S3 SDK, pre-signed uploads from the browser	Free egress; keeps images off the VPS.
Observability	Sentry (FE+BE), Spring Actuator health, UptimeRobot ping, structured JSON logs	Free tiers are enough for year 1.
CI/CD	GitHub Actions → build image → SSH deploy (docker compose pull && up); Vercel auto-deploy for FE	Zero-cost, 5-minute deploys.

4. Module Boundaries

Each module is a top-level package (`lk.ceylonpick.<module>`) exposing a small public API (service interfaces + DTOs + events). Other modules may depend only on that API, never on entities or repositories. Spring Modulith's `ApplicationModules.verify()` runs in CI to enforce this.

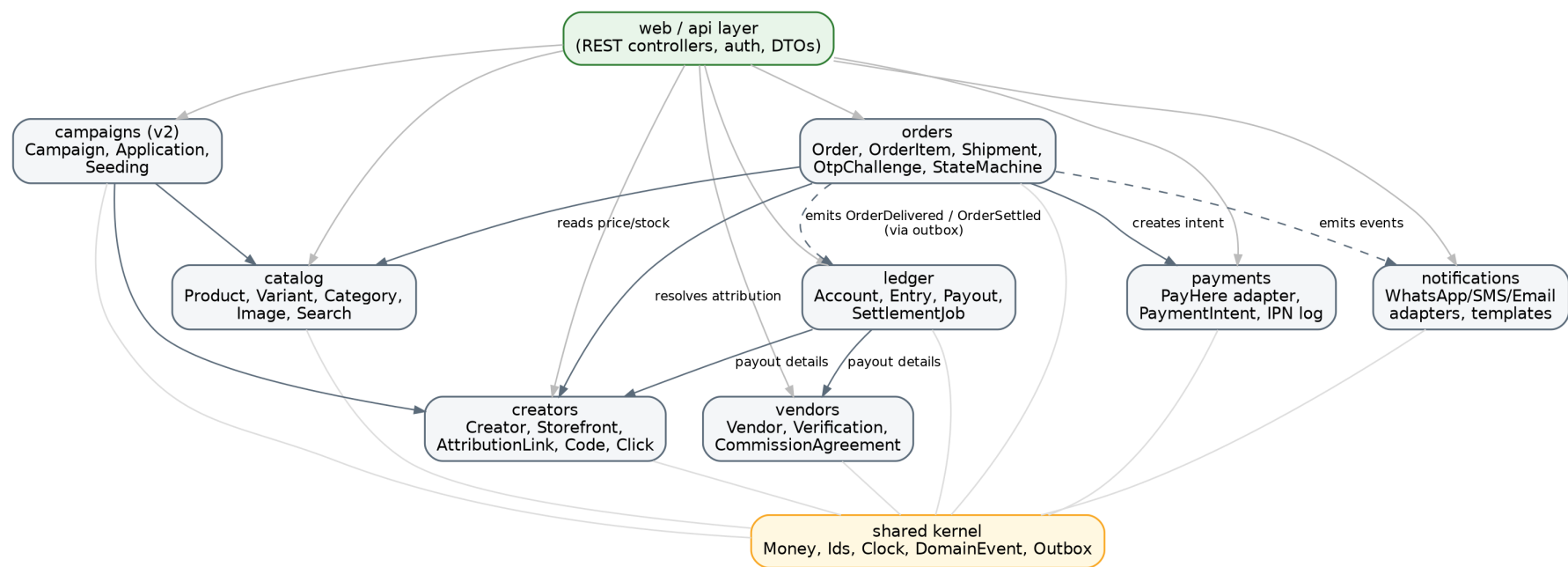


Figure 3 — Module map. Solid arrows = synchronous API calls; dashed = domain events via outbox. Money-critical modules are orders, payments and ledger.

Module	Owns	Public API (examples)	Emits events	May call
catalog	Product, ProductVariant, Category, ProductImage	getProduct(slug), searchProducts(q, category), reserveStock(variantId, qty), releaseStock()	ProductPublished, StockLow	—
vendors	Vendor, VerificationRecord, CommissionAgreement, BankDetailsRef	getVendor(id), listPayableVendors(), verificationQueue()	VendorVerified, VendorSuspended	—
creators	Creator, Storefront, AttributionLink, Click, CreatorStats	resolveAttribution(code cookie), recordClick(), getStorefront(handle), recomputeRtoRate(creatorId)	CreatorApproved, CreatorPaused	—

Module	Owns	Public API (examples)	Emits events	May call
orders	Order, OrderItem, Shipment, OtpChallenge, Dispute, OrderStateMachine	placeOrder(cmd), confirmOtp(orderId, code), markPacked(orderId, photoKey), markShipped(orderId, tracking), applyCourierStatus(), openDispute()	OrderPlaced, OrderConfirmed, OrderShipped, OrderDelivered, OrderSettled, OrderRto, OrderCancelled, OrderRefunded	catalog, creators, payments
payments	Payment, PaymentIntent, IpnLog, PayHereAdapter	createIntent(order), handleIpn(payload), verifySignature()	PaymentSucceeded, PaymentFailed	—
ledger	LedgerAccount, LedgerEntry, Transaction, Payout, SettlementJob, RemittanceImport	post(txn), balance(accountId), payoutPreview(period), runPayout(), importCourierRemittance(csv)	PayoutExecuted	vendors, creators (read bank details ref)
notifications	Template, MessageLog, WhatsAppAdapter, SmsAdapter	send(template, to, vars) — consumed via events only	MessageFailed	—
campaigns (v2)	Campaign, CampaignApplication, Seeding	postCampaign(), apply(), approve()	CampaignApproved	creators, catalog
shared kernel	Money (BigDecimal, LKR, scale 2), Ids (ULID), Clock, DomainEvent, OutboxEntry, IdempotencyKey	—	—	—

Boundary rules

- No module reads another module's tables. Cross-module reads go through the API; if that is too slow, add a read-model, not a join.
- **orders** is the only module that changes order state. **ledger** is the only module that writes ledger entries. **payments** is the only module that talks to PayHere.
- Events are persisted in the outbox in the same transaction as the state change, then dispatched by a poller (at-least-once). Every consumer is idempotent on event_id.

5. Domain Model

Simplified entity model. IDs are ULIDs (sortable, URL-safe). All money is LKR, NUMERIC(12, 2). All timestamps are UTC; display is Asia/Colombo.

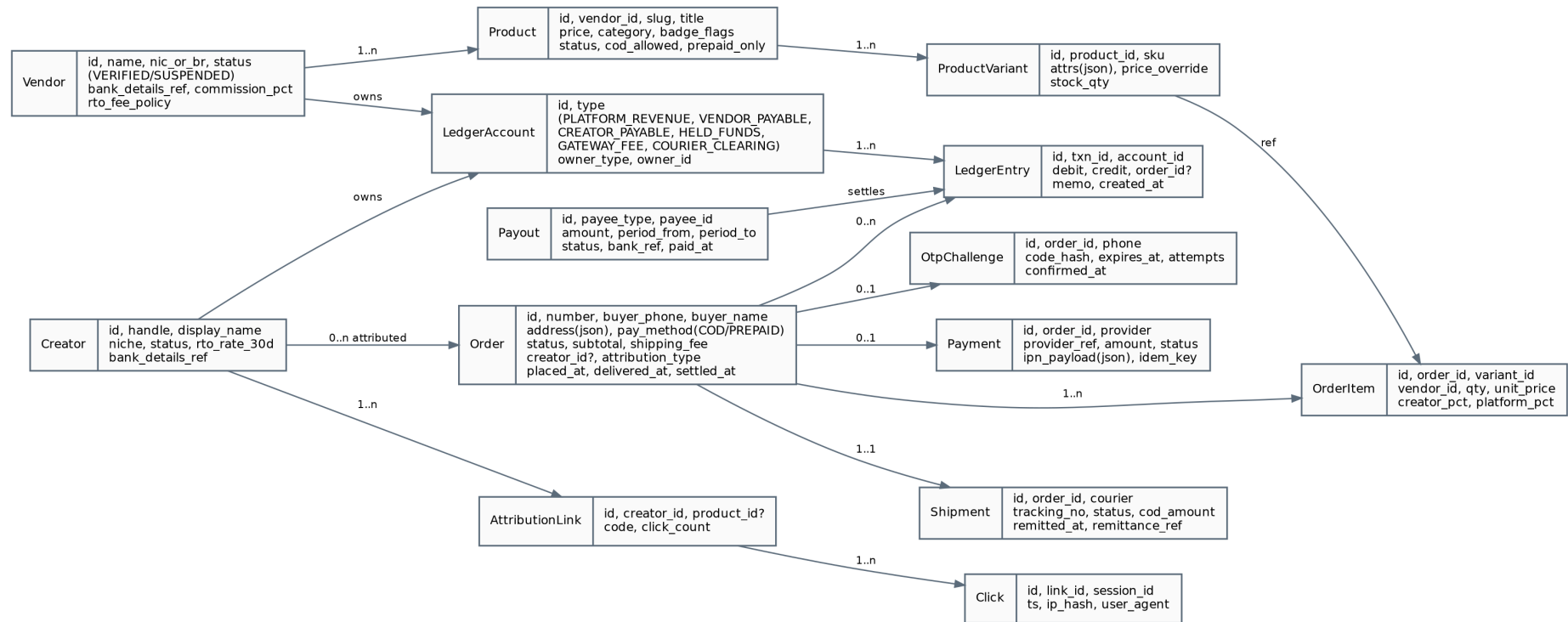


Figure 4 — Domain model (simplified). Bank details are stored as an encrypted reference, never in plain columns.

Entity	Notes that matter for correctness
Order	status is an enum governed by the state machine (Section 6). creator_id and attribution_type are frozen at placement. version column for optimistic locking.
OrderItem	Snapshots unit_price, creator_pct, platform_pct at placement — later vendor changes never alter historic orders.
Shipment	One per order in v1 (single-vendor carts enforced at checkout; multi-vendor carts are split into separate orders).
OtpChallenge	code_hash only (SHA-256 + salt), 6 digits, 10-minute expiry, max 3 attempts, max 3 resends/order.
Payment	idem_key = PayHere payment_id; unique index guarantees duplicate IPNs are no-ops.

Entity	Notes that matter for correctness
LedgerAccount / LedgerEntry	Append-only. Balances are computed (or maintained in a materialised view), never edited. A Transaction groups entries; sum(debit) = sum(credit) enforced by a deferred constraint trigger.
Payout	Groups settled entries for a payee over a period; status DRAFT → APPROVED → PAID; bank reference recorded manually in v1.
Creator.rto_rate_30d	Recomputed nightly from attributed orders; drives auto-pause (>30%) and removal review (>40%).

6. Order State Machine

Order status is the spine of the system: it decides when stock is reserved, when a courier is called, when money moves and when a creator gets paid. Transitions are implemented in one class (`OrderStateMachine`) with an explicit transition table; any attempt outside the table throws.

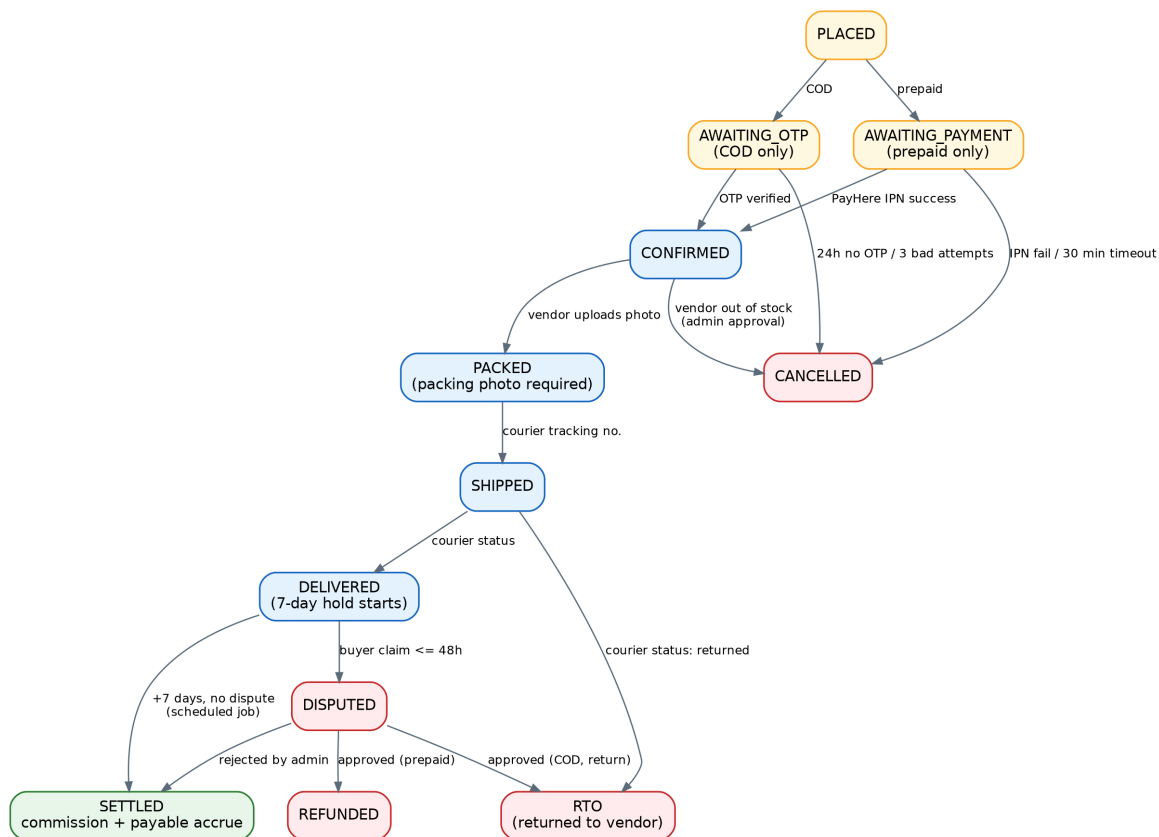


Figure 5 — Order state machine. Terminal states: SETTLED, CANCELLED, RTO, REFUNDED.

From	To	Trigger (actor)	Guards	Side effects (via events)
—	PLACED	Buyer submits checkout	Cart is single-vendor; stock available; COD allowed for all items and total ≤ COD cap	Reserve stock (15 min TTL for prepaid, 24h for COD); freeze attribution
PLACED	AWAITING_PAYMENT	System (prepaid)	—	Create PaymentIntent; redirect to PayHere
PLACED	AWAITING_OTP	System (COD)	—	Send OTP via WhatsApp/SMS
AWAITING_PAYMENT	CONFIRMED	PayHere IPN status=2	Signature valid; amount matches; idem_key unseen	Notify vendor + buyer; ledger: Dr GATEWAY_CLEARING / Cr HELD_FUNDS
AWAITING_PAYMENT	CANCELLED	IPN failed, or 30-min job	—	Release stock
AWAITING_OTP	CONFIRMED	Buyer enters OTP	Code matches, not expired, attempts < 3	Notify vendor + buyer
AWAITING_OTP	CANCELLED	24h job or 3 failed attempts	—	Release stock; count as 'unconfirmed' for creator (not RTO)
CONFIRMED	PACKED	Vendor	Packing photo uploaded	Notify admin for pickup batch
CONFIRMED	CANCELLED	Vendor request + admin approval	Reason recorded	Release stock; refund if prepaid; vendor strike

From	To	Trigger (actor)	Guards	Side effects (via events)
PACKED	SHIPPED	Vendor/admin enters tracking no.	Courier accepted	Notify buyer with tracking
SHIPPED	DELIVERED	Courier status (webhook/poll) or admin	—	Start 7-day hold; COD: Dr COURIER_CLEARING / Cr HELD_FUNDS; notify buyer (dispute window 48h)
SHIPPED	RTO	Courier status returned	—	Charge return fee per vendor policy; increment creator RTO; stock back to vendor
DELIVERED	SETTLED	Settlement job at delivered_at + 7d	No open dispute	Ledger split: VENDOR_PAYABLE / CREATOR_PAYABLE / PLATFORM_REVENUE / GATEWAY_FEE
DELIVERED	DISPUTED	Buyer ≤ 48h after delivery	Photo evidence attached	Pause settlement; notify admin + vendor
DISPUTED	SETTLED / REFUNDED / RTO	Admin decision	Decision reason recorded	As per target state

Implementation notes

- Persist every transition in order_status_history (order_id, from, to, actor, reason, ts). This table powers the buyer's tracking timeline and the admin audit view.
- Use SELECT ... FOR UPDATE on the order row inside each transition; combine with the JPA @Version column.
- Scheduled jobs (OTP expiry, payment timeout, settlement) read due rows in batches of 100 and are guarded by ShedLock so a restart cannot double-run them.

7. Money: Double-Entry Ledger

Customer money is **held funds**, not revenue. The ledger makes that structurally true: every rupee that enters the business is credited to HELD_FUNDS and can only leave through a balanced transaction. The bank statement must equal HELD_FUNDS + PLATFORM_REVENUE – payouts at all times; the Friday reconciliation checks exactly that.

Accounts

Account type	Owner	Meaning
BANK	platform	Actual cash in the business account (mirrors bank statement)
HELD_FUNDS	platform	Buyer money received but not yet split (escrow)
COURIER_CLEARING	per courier	COD collected by courier, not yet remitted to bank
GATEWAY_CLEARING	per gateway	Prepaid amount captured by PayHere, not yet settled to bank
GATEWAY_FEE	platform	PayHere fees (expense; charged to vendor share in v1 pricing)
VENDOR_PAYABLE	per vendor	Owed to vendor after settlement
CREATOR_PAYABLE	per creator	Owed to creator after settlement
PLATFORM_REVENUE	platform	CeylonPick commission — the only revenue line
RTO_FEES	platform	Return-shipping recoveries charged to vendors (offsets courier expense)

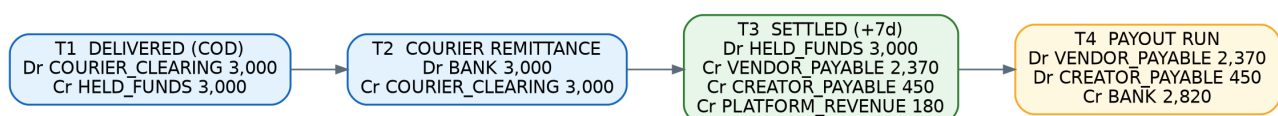


Figure 6 — Worked example: a LKR 3,000 COD order with 15% creator commission and 6% platform commission.

Schema (Flyway V1)

```
CREATE TABLE ledger_account (
```

```

id          TEXT PRIMARY KEY,          -- ULID
type        TEXT NOT NULL,             -- BANK | HELD_FUNDS | COURIER_CLEARING | ...
owner_type  TEXT NOT NULL,             -- PLATFORM | VENDOR | CREATOR | COURIER | GATEWAY
owner_id    TEXT,                      -- nullable for PLATFORM
currency    CHAR(3) NOT NULL DEFAULT 'LKR',
            UNIQUE (type, owner_type, owner_id)
);
CREATE TABLE ledger_txn (
id          TEXT PRIMARY KEY,
kind        TEXT NOT NULL,             -- COD_DELIVERED | PREPAID_CAPTURED | REMITTANCE | SETTLEMENT | PAYOUT | REFUND | RTO
order_id    TEXT,                      -- nullable (payouts, remittances span orders)
payout_id   TEXT,
idem_key    TEXT UNIQUE,               -- e.g. 'SETTLEMENT:<order_id>' prevents double posting
memo        TEXT,
created_by  TEXT NOT NULL,             -- SYSTEM | ADMIN:<id>
created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE ledger_entry (
id          BIGSERIAL PRIMARY KEY,
txn_id      TEXT NOT NULL REFERENCES ledger_txn(id),
account_id  TEXT NOT NULL REFERENCES ledger_account(id),
debit       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (debit >= 0),
credit      NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (credit >= 0),
CHECK ((debit = 0) <> (credit = 0))      -- exactly one side non-zero
);
-- Balanced-transaction guard (deferred constraint trigger, checked at COMMIT):
-- SELECT SUM(debit) - SUM(credit) FROM ledger_entry WHERE txn_id = NEW.txn_id must equal 0.
CREATE INDEX ON ledger_entry (account_id, id);
CREATE MATERIALIZED VIEW account_balance AS
SELECT account_id, SUM(credit) - SUM(debit) AS balance FROM ledger_entry GROUP BY account_id;

```

Rules

- Never UPDATE or DELETE a ledger row. Corrections are new ADJUSTMENT transactions with a memo and admin id.
- Settlement of an order is one transaction with idem_key SETTLEMENT:<order_id>; re-running the job is safe.
- Payout run = preview (sum of payable per payee with balance $\geq$ minimum) → admin approves → CSV for bank bulk transfer → mark PAID with bank ref → PAYOUT transactions posted. v1 does not integrate with bank APIs.
- Courier remittance CSV import creates one REMITTANCE transaction per remittance batch and reconciles against COURIER_CLEARING; unmatched shipments appear on the admin exceptions list.
- Rounding: compute creator and platform shares per *order item* with HALF_EVEN to 2 dp; vendor share = item total – creator – platform – gateway fee (absorbs rounding).

8. Creator Attribution

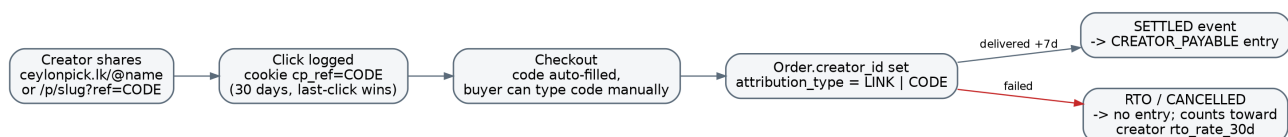


Figure 7 — Attribution flow. Commission is only ever recognised on SETTLED.

Rule	Value	Reason
Cookie window	30 days, last-click wins	Matches Daraz/Amazon norms creators already understand
Manual code at checkout	Overrides cookie	Creators promote codes verbally in videos; codes must beat stale cookies
Self-referral	Blocked by phone number match (creator's registered phone $\neq$ buyer phone)	Basic fraud control
Attribution frozen	At PLACED	Prevents disputes about who gets paid
Commission recognised	At SETTLED only (delivered + 7d, no dispute)	Kills fake-COD fraud
RTO/unconfirmed impact	Counted in creator rto_rate_30d; unconfirmed OTP counted separately	Distinguishes bad traffic from bad buyers

Rule	Value	Reason
Auto-pause	rto_rate_30d > 30% with ≥ 10 orders	Small samples do not trigger
Storefront	/@handle lists creator picks; product pages carry ?ref=CODE	One link creators can put in bio

9. Integrations

9.1 PayHere

- Hosted checkout (redirect). CeylonPick never touches card data → PCI scope is SAQ-A.
- Server generates the request hash: `MD5(merchant_id + order_id + amount(2dp) + currency + MD5(secret).upper()).upper()`. Amount string formatting must be exact.
- IPN handler (POST /webhooks/payhere): verify `md5sig`, verify amount and currency against the intent, upsert Payment by `payment_id` (idempotent), then call `orders.confirmPayment()`. Respond 200 quickly; do work inside a transaction, not in the HTTP thread beyond that.
- Never confirm an order from the browser return URL — only from IPN. Show 'payment processing' until IPN lands (usually seconds).
- Reconciliation job: daily compare Payments SUCCEEDED vs PayHere settlement report; exceptions to admin list.
- Sandbox for all development; live merchant needs registered business.

9.2 Courier

- CourierPort interface: `createShipment(order) → trackingNo, fetchStatus(trackingNo), parseRemittance(csv)`. One adapter per courier; a `ManualCourierAdapter` lets admin type tracking numbers and statuses when APIs are unavailable (v1 reality).
- Status mapping table per courier → internal enum {IN_TRANSIT, DELIVERED, RETURNED, FAILED_ATTEMPT}. Two failed attempts trigger a WhatsApp nudge to the buyer.
- Poll every 2 hours for SHIPPED orders older than 12h if no webhook is offered.

9.3 Messaging (OTP and notifications)

- MessagingPort with WhatsApp Business API adapter (template messages) and SMS fallback via a local gateway. OTP goes SMS-first for reliability; status updates WhatsApp-first for cost.
- All sends are event-driven (OrderConfirmed → 'Your order is confirmed'); the notifications module owns templates in EN/SI/TA and logs delivery status.
- Rate limits: max 3 OTPs per phone per hour; max 1 marketing message per buyer per week.

9.4 Storage

- Browser requests a pre-signed PUT URL; uploads go directly to R2; server stores the object key. Images are resized client-side to ≤ 1600px before upload and served via Cloudflare with width variants.
- Packing photos are private objects; served through short-lived signed URLs to admin and the buyer on dispute.

10. Dashboards: How Many, and What Goes on Each

Three dashboards plus one lightweight buyer account. Each dashboard has **one primary number** at the top, three to four supporting numbers, and an action list. Tables come after numbers. Anything that does not lead to an action this week is not on the dashboard.

Dashboard	Primary user	Primary number (top of screen)	Supporting numbers	Actions available	v1 / v2
A. Ops & KPI (Admin)	Founder	Delivered orders this week vs target	RTO % (7d/30d) · Contribution margin/order · Creator payouts due · Held funds vs bank	Verify vendor · Approve creator · Resolve dispute · Approve payout run · Import remittance CSV · Mark shipped/delivered (manual courier)	v1
B. Vendor	Vendor owner	Orders to pack today	Payable balance · Next payout date · Stock-low items · RTO % on my orders	Upload packing photo · Enter tracking · Edit stock/price · Request cancel · Download payout statement	v1 (payout statement v1, analytics v2)
C. Creator	Creator	Payable balance (and next payout date)	Clicks (7d) · Confirmed orders · Delivered · RTO % · Top product	Copy link/code · Edit storefront picks · Apply to campaign (v2) · Download statement	v1 basic (balance, links, orders); charts + Campaign Board v2
D. Buyer account (not a dashboard)	Buyer	Where is my order (timeline)	—	Track · Confirm OTP · Open dispute (≤48h) · Reorder	v1

10.1 Admin Ops & KPI dashboard — layout

Row 1 (numbers): Delivered this week / target · RTO 7d · Margin per order · Payouts due Friday. Row 2 (queues, each a count that is a button): Awaiting pickup · OTP unconfirmed > 12h · Disputes open · Vendors awaiting verification · Creators awaiting approval · Remittance exceptions. Row 3 (tables): Orders (filter by status, vendor, creator) and Ledger (held funds vs bank check with green/red badge). Weekly review (Monday) is done entirely from this screen; if a number is not here, it is not a KPI.

10.2 Vendor dashboard — layout

Top: 'Orders to pack today' as a large number with a 'Start packing' button that opens a full-screen card flow — one order per card, photo capture → confirm → next. Below: payable balance with the next payout date in words ('Friday 12 Sep'). Then a simple list of products with stock and a tap-to-edit quantity. No charts in v1.

10.3 Creator dashboard — layout

Top: payable balance and next payout date — this is the trust screen. Then a 'Share' block: storefront link, personal code, copy buttons, and a WhatsApp share button. Then a funnel row: Clicks → Confirmed → Delivered → Paid, each with a 7-day count. Then an order list with status chips (same colours as the state machine) and a plain-language RTO note when the rate rises above 20%.

Design principle for all three: a creator or vendor on a phone with 30 seconds should get the answer to 'how much am I owed and what do I need to do today' without scrolling.

11. User-Friendly Design Patterns

11.1 Buyer experience (mobile-first, low-trust market)

Pattern	How it is applied in CeylonPick	Why it matters here
Trust before price	Verified badge, vendor's name and photo, 'Made in ' line above the price on every product card	Buyers distrust unknown sellers; provenance is the product
Creator context banner	When arriving via a creator link: 'Picked by @name' banner persists through checkout	Reassures buyer; increases conversion and attribution clarity

Pattern	How it is applied in CeylonPick	Why it matters here
Single-column, one-action screens	Each checkout step has one primary button; no side-by-side columns on mobile	Reduces abandonment on small screens
Guest checkout by phone	Phone number is the identity; no account creation, no password; OTP doubles as login	Sri Lankan buyers abandon at 'create account'
COD as a first-class choice	COD shown with equal weight; 'We will send a 6-digit code to confirm' explained before it happens	Surprise OTP screens cause drop-off; explained ones don't
Honest shipping line	Courier fee shown early (on product page), not at the last step	Late fee reveal is the #1 checkout complaint
Status chips & timeline	Same colours as the state machine everywhere (amber = waiting on you, blue = in progress, green = done, red = problem)	Consistency lowers support messages
Progressive disclosure	Product page: image, price, badge, buy button first; description, vendor story and policies below the fold	Speed to decision on 3G/4G
Performance budget	LCP < 2.5s on a mid-range Android over 4G; images ≤ 120 KB; no client-side JS on product pages beyond the buy button	Most traffic is mobile on variable networks
Language toggle	EN / ■■■■■■ / ■■■■■■ on header; product content in vendor's language with optional translations	Reach outside English-speaking Colombo
WhatsApp as the help desk	Floating WhatsApp button; order status messages come from the same number	Buyers already live in WhatsApp
Empty and error states	Every list has a designed empty state with a next action; every error names the fix ('Code expired — resend')	Avoids dead ends that cause abandonment

11.2 Dashboard UX patterns (vendor, creator, admin)

- **Numbers first, tables second.** Top row is always cards with a single large number and a one-line comparison ('vs last week').
- **Queues as buttons.** Any count that needs action is tappable and opens the filtered list. No separate menu navigation needed.
- **Card-per-task flows on mobile** (pack this order → photo → next) instead of tables with tiny buttons.
- **Plain-language money.** 'You will receive LKR 12,450 on Friday 12 Sep' beats 'Payable balance: 12450.00'.
- **Explain every deduction.** Statement rows show product price → creator commission → platform fee → your amount, per order.
- **Never surprise on payouts.** The payout calendar is visible on every dashboard; a countdown replaces a support ticket.
- **Optimistic UI only for reversible actions;** money and status changes wait for the server and show a confirmation.
- **Accessibility baseline:** 44px tap targets, 4.5:1 contrast, labels not placeholders, no colour-only meaning (chips carry text).

11.3 Design system

Tailwind + shadcn/ui components; tokens for colour (brand teal, navy, state colours), spacing (4px grid), radius (8px), and typography (Inter for Latin; Noto Sans Sinhala and Noto Sans Tamil loaded on demand). Component inventory for v1: ProductCard, VerifiedBadge, CreatorBanner, StatusChip, OrderTimeline, NumberCard, QueueButton, PhotoCapture, PayoutStatementRow, EmptyState. Storybook is optional; a single /design route rendering all components is enough for one developer.

12. Eight-Week Build Plan

Money-critical paths are built first and tested first. Front-end polish comes last. Each week ends with a deployable increment.

Week	Back end (Spring)	Front end (Next.js)	Exit criteria
1	Repo, Docker Compose (Postgres, app, Caddy), Flyway V1 (all tables), shared kernel (Money, ULID, Outbox), auth skeleton, CI	Project scaffold, design tokens, component inventory page, PWA manifest	Deploys to VPS; health endpoint green; login works
2	catalog + vendors modules; image upload via R2 pre-signed URLs; admin vendor verification	Home, category, product page (SSR/ISR), vendor 'Sell with us' form	20 real products listed with photos
3	orders module: place order, stock reservation, OTP challenge, state machine + history; notifications adapter (SMS OTP)	Checkout (COD path), OTP screen, order tracking page	End-to-end COD order confirmed by OTP on a real phone
4	payments module: PayHere intent, IPN handler, timeout job; refund stub	Checkout (prepaid path), payment pending/success screens	Sandbox prepaid order reaches CONFIRMED via IPN only
5	ledger module: accounts, balanced-txn trigger, settlement job (+7d), remittance CSV import, payout preview/run	Admin Ops & KPI dashboard (numbers + queues + orders table + ledger check)	Worked example (Fig. 6) reproduces exactly in the ledger; bank-vs-held check passes
6	creators module: links, clicks, cookie/code attribution, rto_rate job, auto-pause	Creator storefront /@handle, 'Earn with CeylonPick' page + application, creator dashboard (balance, links, orders)	Attributed order settles into CREATOR_PAYABLE
7	Shipment + courier manual adapter, dispute flow, WhatsApp status templates, Sentry, backups to R2, ShedLock	Vendor dashboard (pack flow with photo capture, tracking entry, payable), dispute form for buyers	Full lifecycle PLACED → SETTLED and PLACED → RTO exercised with real courier
8	Load test (100 concurrent), security pass (rate limits, OWASP top 10 checklist), data seed for 20 vendors	Performance budget met (LCP < 2.5s), SI/TA strings, empty states, polish	Go-live checklist signed; 20 vendors, 20 creators onboarded

Go-live checklist

- PayHere live merchant approved; IPN URL reachable over HTTPS; sandbox and live secrets separated.
- Nightly pg_dump to R2 verified by a restore test.
- Admin can manually move any order between allowed states with a reason (courier APIs will fail).
- Payout CSV export tested against the bank's bulk-transfer template.
- Kill switch: environment flag to disable new orders while keeping tracking and dashboards live.

If week 8 arrives and the exit criteria for weeks 3, 4 and 5 are not all met, stop and ship on WooCommerce + Dokan + PayHere plugin. The custom build can resume after the day-90 review.

13. Non-Functional Requirements

Area	Requirement (year 1)
Performance	Product page TTFB < 300 ms from cache; checkout API p95 < 500 ms; dashboards p95 < 1 s. Target load: 50 orders/hour peak, 5,000 daily visitors.
Availability	99.5% (≈ 3.6 h/month). Single VPS accepted; recovery = redeploy from image + restore dump (RTO 2 h, RPO 24 h).
Security	HTTPS only; JWT in httpOnly/SameSite cookies; CSRF for cookie auth; rate limiting per IP and phone (bucket4j in-process); secrets in env/Docker secrets; dependency scanning in CI; no card data ever stored; bank details encrypted at rest (AES-GCM, key outside DB).
Privacy	Collect only phone, name, address. Retention: clicks 90 days, IP hashed, buyer data purge on request. Privacy notice in three languages.
Auditability	order_status_history, ledger append-only, admin actions logged with actor id.

Area	Requirement (year 1)
Observability	Sentry errors; Actuator metrics scraped weekly (orders by status, job lag, IPN failures); daily digest email to founder.
Backups	Nightly pg_dump + weekly full VPS snapshot; R2 versioning on.
Testing	Unit tests for state machine (every transition, every guard) and ledger (balanced txns, idempotency); integration tests with Testcontainers Postgres; one end-to-end Playwright flow for COD and prepaid.

14. Architecture Decision Records

ADR	Decision	Alternatives rejected	Revisit when
ADR-1	Modular monolith on one Spring Boot deployable	Microservices; serverless functions	Team ≥ 3 engineers or a module needs independent scaling
ADR-2	PostgreSQL only; no Redis, no broker; outbox table for events	Redis cache/sessions; RabbitMQ/Kafka	2+ app instances, > 1,000 orders/day, or job lag > 5 min
ADR-3	Next.js PWA, no native app in year 1	Flutter/React Native app	Month 12 review with repeat rate > 25% and > 2,000 monthly buyers
ADR-4	PayHere hosted checkout; confirm via IPN only	Direct card API; confirming on return URL	Never confirm on return URL. Add second gateway if PayHere approval rate < 90%
ADR-5	Double-entry ledger with append-only entries	Balance columns on vendor/creator rows	Never
ADR-6	Order status as explicit state machine with history table	Ad-hoc status updates in services	Never
ADR-7	Single-vendor orders (carts split per vendor)	Multi-vendor orders with partial shipments	When > 20% of carts contain 2+ vendors
ADR-8	Manual courier adapter first; APIs when stable	Wait for courier API integration before launch	Courier offers a documented API with webhooks
ADR-9	Phone-OTP passwordless auth for buyers	Email/password accounts	Never for buyers
ADR-10	Custom build with 8-week timebox; WooCommerce as fallback	WooCommerce from day 1	Week 8 if exit criteria for weeks 3–5 unmet

15. Open Items for the Next Version

- Confirm which courier partner provides a status API or webhook; update Section 9.2 and the manual-adapter scope.
- Confirm WhatsApp Business API access and template approval lead time; otherwise SMS-only for v1 notifications.
- Decide vendor RTO fee policy (fixed LKR 250 vs 50/50 split) — affects ledger RTO_FEES postings.
- Campaign Board (v2) domain model and permissions.
- Buyer referral codes (v2): reuse AttributionLink with owner_type = BUYER, or separate entity.
- Signature collection (v3) placement fee: subscription posting pattern in ledger (monthly PLATFORM_REVENUE, no order_id).

End of document — v1.0